

EXPERIMENT 3

Title

Using the Employee schema, write and execute queries using INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare query execution plans using EXPLAIN.

Aim

To understand and implement advanced SQL query techniques on the Employee schema, including different types of joins, subqueries, set-operation simulations, and execution-plan analysis using EXPLAIN.

Theory

SQL joins and subqueries are used to retrieve related information from one or more tables. A join combines rows based on a related column, while a subquery is a query written inside another query. Execution plans show how the database engine intends to access and process data.

- INNER JOIN returns only rows that have matching values in both tables.
- LEFT JOIN returns all rows from the left table and matching rows from the right table. If there is no match, NULL values are returned.
- Self-join joins a table with itself. It is useful for hierarchical relationships such as employees and their managers.
- A 3-way join combines three related tables in one query.
- A correlated subquery depends on a value from the outer query and is evaluated with reference to the current outer row.
- EXISTS checks whether a subquery returns at least one row.
- INTERSECT returns common rows between two result sets. In MySQL, it can be simulated using INNER JOIN or EXISTS when needed.
- EXCEPT returns rows from the first result set that are not present in the second. In MySQL, it can be simulated using NOT EXISTS or LEFT JOIN with IS NULL.
- EXPLAIN displays the execution plan, including access type, possible keys, chosen key, estimated rows, and extra operations.
- Indexes can change the execution plan by allowing the optimizer to access rows more efficiently.

Employee Schema Used

```
Department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50),  
    location VARCHAR(50)  
)
```

```
Employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(60),
```

```

        job_title VARCHAR(50),
        salary DECIMAL(10,2),
        hire_date DATE,
        department_id INT,
        FOREIGN KEY (department_id) REFERENCES Department(department_id)
    )

Project(
    project_id INT PRIMARY KEY,
    project_name VARCHAR(80),
    budget DECIMAL(12,2),
    start_date DATE,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
)

Employee_Project(
    employee_id INT,
    project_id INT,
    hours_worked INT,
    role VARCHAR(50),
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
)

```

SQL Queries and Demonstration

1. INNER JOIN

```

SELECT e.employee_id,
       e.employee_name,
       e.job_title,
       d.department_name
FROM Employee e
INNER JOIN Department d
    ON e.department_id = d.department_id;

```

2. LEFT JOIN

```

SELECT d.department_id,
       d.department_name,
       e.employee_name
FROM Department d
LEFT JOIN Employee e
    ON d.department_id = e.department_id
ORDER BY d.department_id;

```

3. SELF-JOIN

```

ALTER TABLE Employee
ADD COLUMN manager_id INT NULL;

SELECT e.employee_name AS employee,
       m.employee_name AS manager

```

```

FROM Employee e
LEFT JOIN Employee m
    ON e.manager_id = m.employee_id;

```

4. THREE-WAY JOIN

```

SELECT e.employee_name,
       d.department_name,
       p.project_name,
       ep.role
FROM Employee e
INNER JOIN Department d
    ON e.department_id = d.department_id
INNER JOIN Employee_Project ep
    ON e.employee_id = ep.employee_id
INNER JOIN Project p
    ON ep.project_id = p.project_id;

```

5. CORRELATED SUBQUERY

```

SELECT e.employee_id,
       e.employee_name,
       e.salary,
       e.department_id
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department_id = e.department_id
);

```

6. EXISTS

```

SELECT e.employee_id,
       e.employee_name,
       e.job_title
FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.employee_id = e.employee_id
);

```

7. SIMULATED INTERSECT

```

SELECT e.employee_id,
       e.employee_name
FROM Employee e
WHERE e.department_id = 1
    AND EXISTS (
        SELECT 1
        FROM Employee_Project ep
        WHERE ep.employee_id = e.employee_id
    );

```

8. SIMULATED EXCEPT

```
SELECT e.employee_id,  
       e.employee_name  
FROM Employee e  
WHERE e.department_id = 1  
      AND NOT EXISTS (  
        SELECT 1  
        FROM Employee_Project ep  
        WHERE ep.employee_id = e.employee_id  
      );
```

9. SIMULATED INTERSECT USING INNER JOIN

```
SELECT DISTINCT e.employee_id,  
               e.employee_name  
FROM Employee e  
INNER JOIN Employee_Project ep  
      ON e.employee_id = ep.employee_id  
WHERE e.department_id = 1;
```

10. SIMULATED EXCEPT USING LEFT JOIN

```
SELECT e.employee_id,  
       e.employee_name  
FROM Employee e  
LEFT JOIN Employee_Project ep  
      ON e.employee_id = ep.employee_id  
WHERE e.department_id = 1  
      AND ep.employee_id IS NULL;
```

11. EXPLAIN – INNER JOIN

```
EXPLAIN  
SELECT e.employee_id,  
       e.employee_name,  
       d.department_name  
FROM Employee e  
INNER JOIN Department d  
      ON e.department_id = d.department_id;
```

12. EXPLAIN – CORRELATED SUBQUERY

```
EXPLAIN  
SELECT e.employee_id,  
       e.employee_name,  
       e.salary  
FROM Employee e  
WHERE e.salary > (  
  SELECT AVG(e2.salary)  
  FROM Employee e2  
  WHERE e2.department_id = e.department_id  
);
```

13. EXPLAIN – EXISTS

```
EXPLAIN
SELECT e.employee_id,
       e.employee_name
FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.employee_id = e.employee_id
);
```

14. EXPLAIN – LEFT JOIN

```
EXPLAIN
SELECT e.employee_id,
       e.employee_name
FROM Employee e
LEFT JOIN Employee_Project ep
    ON e.employee_id = ep.employee_id
WHERE e.department_id = 1
    AND ep.employee_id IS NULL;
```

Comparison of Execution Plans

EXPLAIN Item	Meaning	What to Observe
type	Access/join method used by MySQL.	Look for efficient access types such as const, eq_ref, ref, or range when appropriate.
possible_keys	Indexes that could potentially be used.	Check whether relevant foreign-key or search-column indexes are available.
key	The index actually selected by the optimizer.	A useful index may reduce the number of rows examined.
rows	Estimated number of rows examined.	Generally, a smaller estimate can indicate less work, but it is only an estimate.
Extra	Additional execution details.	Look for operations such as Using where, Using index, or temporary/filesort where relevant.

Recommended Indexes for Join Columns

```
CREATE INDEX idx_employee_department
ON Employee(department_id);

CREATE INDEX idx_employee_project_employee
ON Employee_Project(employee_id);

CREATE INDEX idx_employee_project_project
ON Employee_Project(project_id);
```

Result

The Employee schema was used successfully to demonstrate INNER JOIN, LEFT JOIN, self-join, multi-table (3-way) joins, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. EXPLAIN was used to inspect and compare the execution strategies of different queries.

Conclusion

Different SQL techniques can produce the required result in different ways. Joins are useful for combining related tables, while subqueries and EXISTS are useful for conditional filtering. Simulated set operations can be implemented with EXISTS, NOT EXISTS, INNER JOIN, or LEFT JOIN. EXPLAIN helps understand how MySQL plans to execute these queries and which indexes or access methods may be used.